PAR Laboratory Assignment
Lab 1: Basics: Tareador Tool

Mario Acosta, Eduard Ayguadé, Rosa M. Badia, Jesús Labarta, Pau Ferrer,
Daniel Jiménez-González, Pablo Vizcaino, and Gladys Utrera

Spring 2025–2026

UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Departament d'Arquitectura de Computadors

Contents

Objectives

At the end of this laboratory you should be able to:

1. Use Tareador, an automatic tool to build the task dependency graph (TDG) for a parallel decomposition strategy.

Chapter 1

Systematically analysing task decompositions with Tareador

In this laboratory session, you will learn how to use Tareador, a tool that analyses the potential parallelism that can be obtained when a given *task decomposition* strategy is applied to your sequential code. This way the programmer simply needs to identify which are the tasks in that strategy that wants to evaluate.

These are the main features of Tareador and Tareador GUI:

- Tareador traces the execution of the program based on the specification of potential tasks to be run in parallel;
- Tareador records all static/dynamic data allocations and memory accesses in order to build the task dependency graph (TDG);
- Tareador GUI uses Tareador output information to simulate the parallel execution of the tasks on a certain number of processors in order to estimate the potential speedup.

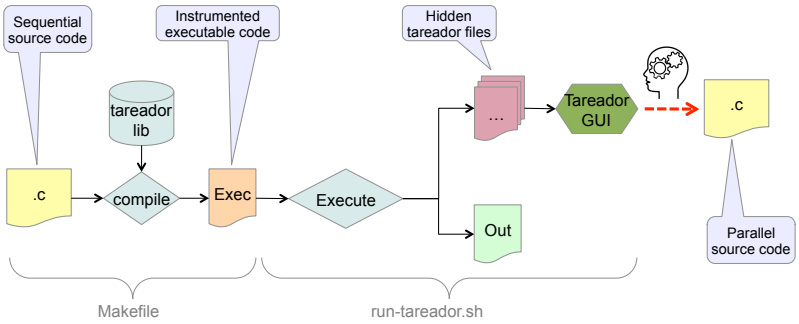

Figure 1.1: Compilation and execution flow for Tareador.

Figure 1.1 shows the compilation and execution flow for Tareador, starting from the taskified source code.

1.1 Tareador API

The Tareador application programmer interface (API) allows to specify code regions *to be considered* as potential tasks:

```
tareador_start_task("TaskName");
/* Code region to be a potential task */
tareador_end_task("TaskName");
```

where the string `TaskName` identifies that task in the graph produced by Tareador. In order to enable the analysis with Tareador, the programmer must invoke:

```
tareador_ON();
/* Main program */
tareador_OFF();
```

at the beginning and at the end of the program, respectively.

From now onwards you will use a program that computes the fast Fourier transform (FFT) of an input dataset in three directions (x, y, and z), producing an output dataset that can be validated for correctness. We ask you to follow the next steps:

1. Go to the `lab1/3dfft` directory, open the `3dfft_tar.c` source code and identify the calls to the Tareador API; try to understand the tasks that have been initially defined.
2. Open the `Makefile` to understand how the source code is compiled and linked to produce the executable.
3. Generate the executable by running:

   `make 3dfft_tar`
4. Execute the binary generated with:

   `./run-tareador.sh 3dfft_tar`

   Note that, due to the instrumentation overhead, the execution time may be several orders of magnitude higher than that of the original sequential code.
5. A window with a warning message will appear, just click on the `Ok` button to continue with the instrumented execution.

1.2 Brief Tareador hands-on

Now you will go through some of the different options that Tareador offers to analyse the potential of task decomposition strategies.

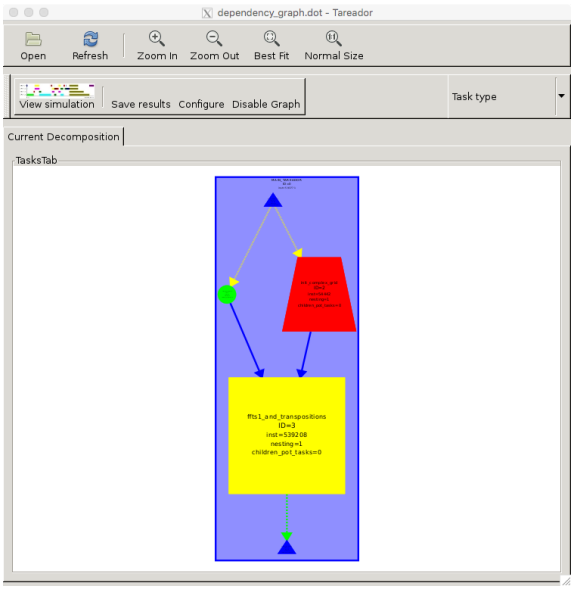

Figure 1.2: Task dependency graph (TDG) for the initial task decomposition expressed in `3dftt_tar.c`.

The execution of the `run_tareador.sh` script opens a new window in which the TDG is visualised (see Figure 1.2):

- each node of the graph represents a task: different shapes and colours are used to identify different task definitions;
- each node is labeled with a task instance number;
- each node contains the number of instructions that the task instance has executed, as an indication of the task granularity;
- the size of the node also reflects in some way this granularity.

You can zoom in and out to see the names of the tasks (the same that were provided in the source code) and the information reported for each node.

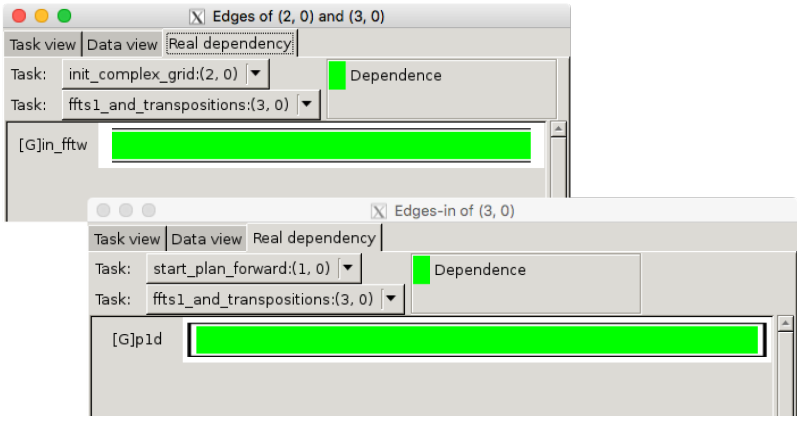

Figure 1.3: Variables provoking data dependencies between tasks for a specific or all edges: real dependency tab.

Edges in the graph represent dependencies between task instances. Different colours/patterns are used to represent different types of dependencies: e.g. blue colour for *data* dependencies and green/yellow for *control* dependencies. Moreover, Tareador allows you to analyse the variables whose access provokes data dependencies between a pair of nodes. Try the following:

1. Move the mouse over an edge. e.g. the edge going from the red task (`init_complex_grid`) to the yellow task (`ffts1_and_transpositions`).
2. Right click with the mouse and select `Dataview` → `edge`. This will open a window similar to the one shown in Fig. 1.3.
3. In the `Real dependency` tab, you can see the variable that causes that dependency (the access to the array `in_fftw`).
4. Alternatively, right click on a task (e.g. `ffts1_and_transpositions`) and select `Dataview` → `Edges-in`. This will open a window similar to the previous one.
5. You can do the same for the edges going out of a task by selecting `Dataview` → `Edges-out` when clicking on top of a task.

For each node you can also analyse the variables that are accessed during the execution of the associated task. For example, with the mouse on node `ffts1_and_transpositions`, carry out the following steps:

1. Right click with the mouse and select `Dataview` → `node`. You can select either the `Task view` tab or the `Data view` tab in that window, as shown in Fig. 1.4.
2. In the `Task view` tab you can see the variables that are:
   - read (i.e. with a `load` memory access) in green colour in the window, as in this case variable `p1d`;
   - written (i.e. with a `store` memory access) in blue colour in the window;
   - both read and written (i.e. `intersection`) in orange, as it is the case for `in_fftw`.

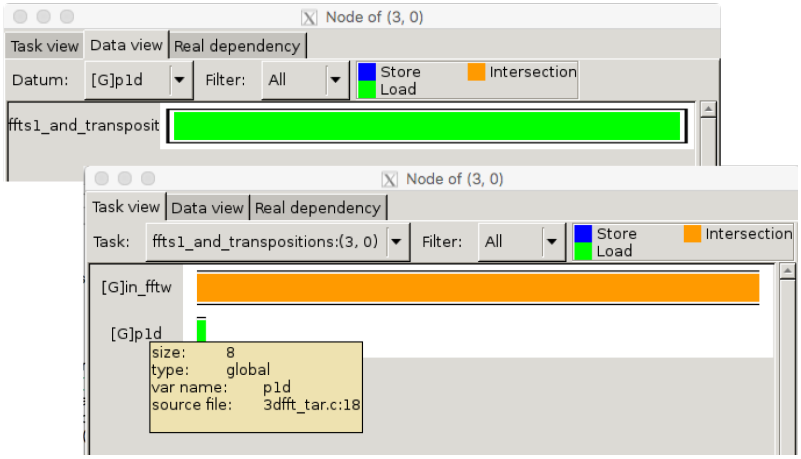

Figure 1.4: Variables provoking data dependencies between tasks for a specific or all edges: dataview and taskview tabs.

3. For each variable in the list you have its name and its storage:
   - `G` for global;
   - `H` for the heap or dynamically allocated data;
   - `S` for the stack or function local variables;

   and additional information (size and allocation) is obtained by placing the mouse on the name.
4. In the `Data view` tab you can see for each variable (selected in the chooser) the kind of access (store, load or both, using the same colors) that are performed by the task.
5. Finally, you can save the TDG by clicking on the `Save results` button in the main Tareador window.
6. Alternatively, you can simply take a screenshot by pressing the `Prtsc` keyboard button.

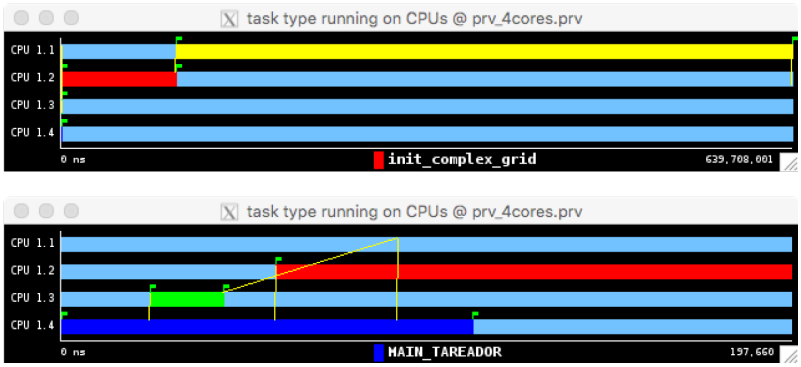

Figure 1.5: Paraver trace of the simulated execution with 4 processors, full view and after zooming into the initial part of the trace.

You can also simulate the execution of the task graph in an *ideal* machine with a certain number of processors by clicking on `View Simulation` in the main window of Tareador GUI. This will open a Paraver window similar to Fig. 1.5 showing the timeline for the simulated execution with the following characteristics:

- each horizontal line shows the task(s) executed by each processor (`CPU1.x`, with `x={1,2,3,4}`);
- colours represent different tasks and match those of the TDG;
- the number on the lower-right corner of the window indicates the simulated execution time (in time units, assuming that each instruction takes one time unit) for the parallel execution;

- yellow lines show task dependencies (and task creations).

You can zoom in the *initial* part of the timeline by pressing the left button of your mouse and selecting the zone you want to zoom. You can undo the zooms done by clicking `Undo zoom` or `Fit time scale` on top of the timeline windows of Paraver. You can save the timeline for the simulated parallel execution by clicking `Save` → `Save image` on top of the timeline Paraver window; alternatively, you can make a screenshot.

1.3 Disabling Tareador objects

Although not useful for this code, you can disable the analysis of certain variables in the program using the following functions of the Tareador API:

```
tareador_disable_object(&VariableName)
/* Code region with memory accesses to 'VariableName' */
tareador_enable_object(&VariableName)
```

With this mechanism you remove all the dependencies caused by the selected variable in a specific code region. For example, if you disable the analysis of the variable `in_fftw`, then you will observe that in the new TDG the tasks `init_complex_grid` and `ffts1_and_transpositions` are executed in parallel.

1.4 Exploring new task decompositions

Once you are familiar with the basic features in Tareador, and motivated by the reduced parallelism obtained in the initial task decomposition (named v0 from now on), you will proceed refining the initial tasks with the objective of discovering more parallelism. You will incrementally generate five new finer–grained task decomposition (named v1, v2, v3, v4 and v5) as described in the following bullets. For each tasks decomposition compute T_1, T_∞ and the potential parallelism ($T_1 \div T_\infty$) from the task dependence graph generated by Tareador, assuming that each instruction takes one time unit to execute. Note that T_∞ can be approximated with a simulation using the maximum number of processors. We advise you to save and compare all the TDGs and fill in Table 1.1.

Now it is time to *refine* the original tasks defined in the FFT benchmark with the objective of exploiting more parallelism. You will incrementally generate five additional fine-grained task decompositions (v1, v2, v3, v4, and v5) as described below:

v1. *REPLACE*[1] the task named `ffts1_and_transpositions` with a sequence of fine-grained tasks, one for each function invocation inside it.

v2. Starting from v1, *REPLACE* the definition of tasks associated to function invocations `ffts1_planes` with fine-grained tasks defined inside the function body and associated to individual iterations of the `k` loop, as shown below:

```
void ffts1_planes(fftwf_plan p1d, fftwf_complex in_fftw[][N][N]) {
  int j, k;
  for (k=0; k<N; k++) {
    tareador_start_task("ffts1_planes_loop_k");
    for (j=0; j<N; j++)
      fftwf_execute_dft(p1d,
                        (fftwf_complex *)in_fftw[k][j][0],
                        (fftwf_complex *)in_fftw[k][j][0]);
    tareador_end_task("ffts1_planes_loop_k");
  }
}
```

For this version pay special attention to the data dependencies that appear in the TDG:

[1] Herein the word "replace" actually means: firstly, remove the *original* task definitions; and, secondly, add the *new* task definitions.

analyze the `Edges-in` for one of the transposition tasks and make sure that you understand what is reported by Tareador.

v3. Starting from v2, *REPLACE* the definition of tasks associated to function invocations `transpose_xy_ planes` and `transpose_zx_planes` with fine-grained tasks inside the corresponding body functions and associated to individual iterations of the `k` loop, as you did in v2 for `ffts1_planes`. Try to understand what is causing data dependencies.

v4. Starting from v3, *REPLACE* the definition of the task for the `init_complex_grid` function with fine-grained tasks inside the body function. For this version, simulate the parallel execution for 1, 2, 4, 8, 16, and 32 processors, drawing a graph or table showing the potential strong scalability. What is limiting the scalability of v4?

v5. Create a new version to explore an even finer granularity. Take into consideration that, due to the large number of tasks, Tareador may take a while to compute and draw the TDG. Once more, simulate the parallel execution for 1, 2, 4, 8, 16, 32, and 128 (i.e. ∞) processors. According to the new results, is it worth going to this granularity level?

Table 1.1: Parallelism for each version of `3dfft_tar.c`.

Version	T_1	T_∞	Parallelism ($T_1 \div T_\infty$)
seq	639Mns	639Mns	1
v1	639Mns	639Mns	1
v2	639Mns	361Mns	1,77
v3	639Mns	154Mns	4,14
v4	639Mns	64Mns	10
v5	639Mns	7Mns	91,3

1.5 Discussion at the laboratory

These questions are to be discussed during and/or at the end of your laboratory session with your professor:

- What is the worst and best parallel strategy according to Table 1.1?
- Which are the main differences between them?
- In a real-world scenario, is the *best strategy* the one that you would have expected to have the *best performance*?